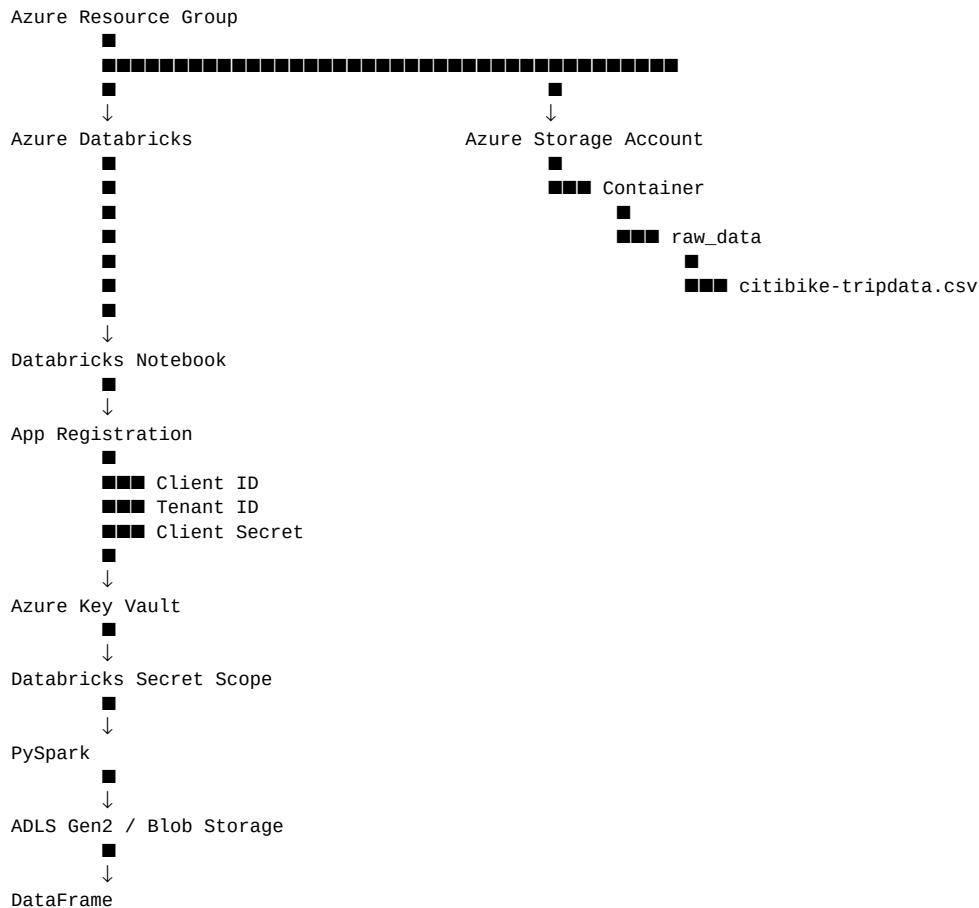


Azure Databricks → Azure Blob Storage / ADLS Gen2

Complete class notes: authentication, Key Vault, Databricks Secret Scope, ABFSS access, and loading CSV data into a PySpark DataFrame.

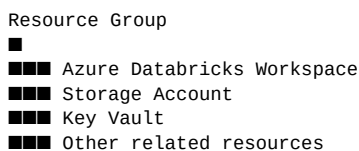
1. Overall Architecture



Main idea: App Registration provides the identity → Key Vault securely stores credentials → Databricks Secret Scope provides secure access to those credentials → Spark authenticates with storage → PySpark reads the CSV into a DataFrame.

2. Step 1 — Create a Resource Group

A Resource Group is a logical container used to organize related Azure resources. For this project, the Databricks workspace, Storage Account, Key Vault, and other related services can be grouped together.

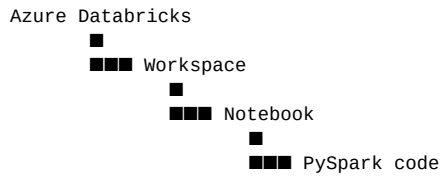


Why create it?

- Organize project resources.
- Manage permissions and access.
- Monitor related resources.
- Manage the project as one logical unit.

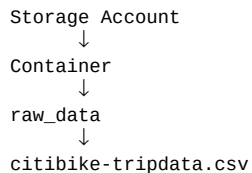
3. Step 2 — Create Azure Databricks Workspace

An Azure Databricks workspace is the environment where data engineering code is developed and executed. A notebook was created inside the workspace for writing and running PySpark code.



4. Step 3 — Create Storage Account and Upload the CSV

An Azure Storage Account was created. Inside it, a container was created and the raw_data folder was used to organize the source file.

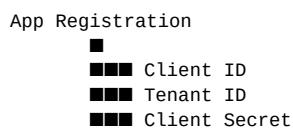


In this class example:

```
Storage Account: blobstoragecitibike
Container: raw-data-container
Folder: raw_data
File: citibike-tripdata.csv
```

5. Step 4 — Create Azure App Registration

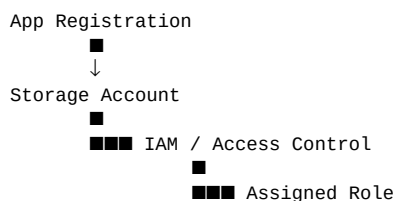
An Azure App Registration was created to provide an application identity for authentication. The important values obtained are Client ID, Tenant ID, and Client Secret.



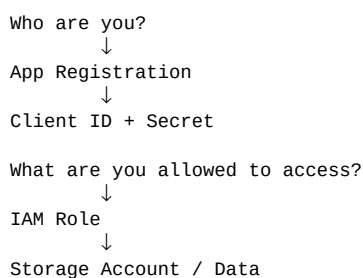
Client ID identifies the application/service principal. Tenant ID identifies the Azure/Entra ID tenant. Client Secret acts as the application credential.

6. Step 5 — Assign IAM Role to the App

Creating an App Registration does not automatically give it access to the Storage Account. The application identity must be assigned an appropriate role through Storage Account → IAM / Access Control.



Authentication vs Authorization



7. Step 6 — Create Azure Key Vault

Azure Key Vault was used to securely store sensitive credentials instead of putting them directly into the Databricks notebook.

```
Azure Key Vault
■
■■■ client-id
■■■ tenant-id
■■■ client-secret
```

Avoid hardcoding sensitive values such as a client secret directly in notebook code. The notebook retrieves them securely through Databricks secrets.

8. Step 7 — Create Databricks Secret Scope

A Databricks Secret Scope provides a secure mechanism for the notebook to retrieve secrets. In the class workflow, the Databricks icon on the top-left was used and **#secrets/createScope** was appended to the workspace URL.

```
Azure Key Vault
■
↓
Databricks Secret Scope
■
↓
Databricks Notebook
```

Scope used in this project:

```
citibike-scope-sp
```

9. Step 8 — Access ADLS Data from Databricks

Once authentication, authorization, Key Vault, and the Databricks Secret Scope are configured, PySpark can authenticate with the storage account and read the Citibike CSV.

10. Complete Code

```
storage_account_name = "blobstoragecitibike"
container_name = "raw-data-container"

client_id = dbutils.secrets.get(scope="citibike-scope-sp", key="client-id")
tenant_id = dbutils.secrets.get(scope="citibike-scope-sp", key="tenant-id")
client_secret = dbutils.secrets.get(scope="citibike-scope-sp", key="client-secret")

# Set authentication on the Spark Session
spark.conf.set(f"fs.azure.account.auth.type.{storage_account_name}.dfs.core.windows.net",
               "OAuth")
spark.conf.set(f"fs.azure.account.oauth.provider.type.{storage_account_name}.dfs.core.windows.net",
               "org.apache.hadoop.fs.azurebfs.oauth2.ClientCredsTokenProvider")
spark.conf.set(f"fs.azure.account.oauth2.client.id.{storage_account_name}.dfs.core.windows.net",
               client_id)
spark.conf.set(f"fs.azure.account.oauth2.client.secret.{storage_account_name}.dfs.core.windows.net",
               client_secret)
spark.conf.set(f"fs.azure.account.oauth2.client.endpoint.{storage_account_name}.dfs.core.windows.net",
               f"https://login.microsoftonline.com/{tenant_id}/oauth2/token")

# Direct access to files/folders
path = f"abfss://{container_name}@{storage_account_name}.dfs.core.windows.net/raw_data"
df = spark.read.format("csv").option("header", "true").load(path)
```

11. Code Explanation — Storage Account and Container

Storage account name:

```
storage_account_name = "blobstoragecitibike"
```

Stores the Azure Storage Account name in a Python variable. It is reused in Spark configuration and in the storage path.

Container:

```
container_name = "raw-data-container"
```

Stores the name of the container inside the Storage Account.

12. Retrieving Secrets with dbutils

These statements retrieve credentials from the Databricks Secret Scope rather than hardcoding them in the notebook.

```
client_id = dbutils.secrets.get(scope="citibike-scope-sp", key="client-id")
tenant_id = dbutils.secrets.get(scope="citibike-scope-sp", key="tenant-id")
client_secret = dbutils.secrets.get(scope="citibike-scope-sp", key="client-secret")
```

The **scope** identifies the secret scope, while the **key** identifies the particular secret.

```
citibike-scope-sp
├─
│   └─ client-id
│       └─ tenant-id
│           └─ client-secret
```

13. Configure Spark Authentication

The Spark configuration statements tell Spark how to authenticate with the ADLS Gen2 endpoint using OAuth and the application identity.

13.1 Authentication Type

```
spark.conf.set(
    f"fs.azure.account.auth.type.{storage_account_name}.dfs.core.windows.net",
    "OAuth"
)
```

Tells Spark that the authentication mechanism is OAuth.

13.2 OAuth Provider

```
spark.conf.set(
    f"fs.azure.account.oauth.provider.type.{storage_account_name}.dfs.core.windows.net",
    "org.apache.hadoop.fs.azurebfs.oauth2.ClientCredsTokenProvider"
)
```

Specifies the OAuth provider that uses client credentials.

13.3 Client ID

```
spark.conf.set(
    f"fs.azure.account.oauth2.client.id.{storage_account_name}.dfs.core.windows.net",
    client_id
)
```

Supplies the application identity Client ID.

13.4 Client Secret

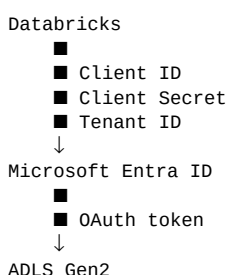
```
spark.conf.set(
    f"fs.azure.account.oauth2.client.secret.{storage_account_name}.dfs.core.windows.net",
    client_secret
)
```

Supplies the Client Secret retrieved securely from the secret scope.

13.5 OAuth Endpoint

```
spark.conf.set(
    f"fs.azure.account.oauth2.client.endpoint.{storage_account_name}.dfs.core.windows.net",
    f"https://login.microsoftonline.com/{tenant_id}/oauth2/token"
)
```

Tells Spark where to obtain the OAuth token; the Tenant ID is inserted dynamically.



14. Build the ABFSS Path

```
path = f"abfss://{container_name}@{storage_account_name}.dfs.core.windows.net/raw_data"
```

This creates the ABFSS path used by Spark to access the folder in ADLS Gen2.

With the class values, the resulting path is:

```
abfss://raw-data-container@blobstoragecitibike.dfs.core.windows.net/raw_data
```

General structure:

```
abfss://container@storageaccount.dfs.core.windows.net/folder
```

Part	Meaning
abfss://	ABFS Secure protocol used to access ADLS Gen2
raw-data-container	Container name
blobstoragecitibike	Storage account name
.dfs.core.windows.net	ADLS Gen2 endpoint
/raw_data	Folder containing source data

15. Read the CSV into a DataFrame

```
df = spark.read \
    .format("csv") \
    .option("header", "true") \
    .load(path)
```

spark.read starts a read operation. **format("csv")** identifies CSV as the source format. **option("header", "true")** tells Spark that the first row contains column names. **load(path)** tells Spark where the data is located.

Example:

```
ride_id,start_time,end_time,start_station
1001,10:00,10:20,Station A
1002,11:00,11:30,Station B
```

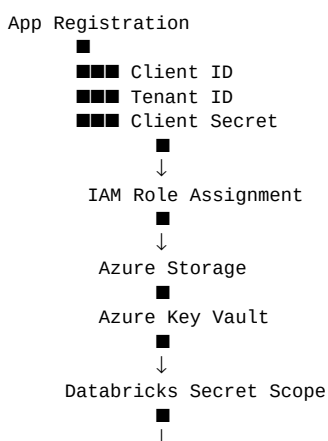
With header=true, Spark uses ride_id, start_time, end_time, and start_station as column names.

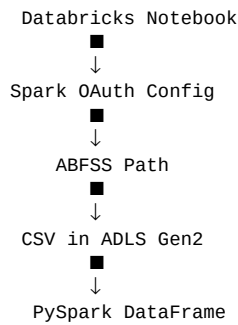
16. Display and Validate the Loaded Data

```
display(df)
df.show()
df.printSchema()
print(df.count())
```

display(df) gives an interactive Databricks table. **df.show()** displays rows. **df.printSchema()** displays column names and data types. **df.count()** returns the number of rows.

17. Authentication Flow — Easy to Remember





18. Complete Step-by-Step Revision

Step	Action	Purpose
1	Create Resource Group	Organize Azure resources
2	Create Databricks workspace + notebook	Develop and run PySpark code
3	Create Storage Account → Container → raw_data → CSV	Store source data
4	Create App Registration	Create application identity
5	Assign IAM role to the app	Authorize storage access
6	Create Key Vault and store credentials	Secure sensitive values
7	Create Databricks Secret Scope	Securely retrieve secrets in notebooks
8	Configure Spark OAuth + ABFSS path	Authenticate and access ADLS data
9	spark.read.csv()	Load CSV into a DataFrame

19. Interview Answer

I created an App Registration for application authentication, granted its identity access to the storage account through IAM, stored its credentials securely in Key Vault, accessed those secrets through a Databricks secret scope, configured Spark for OAuth authentication, and finally used an ABFSS path to read the Citibike CSV data from ADLS Gen2 into a PySpark DataFrame.

20. Important Concepts to Remember

Concept	Remember
App Registration	Provides application identity / service principal
Client ID	Identifies the application
Tenant ID	Identifies the Azure/Entra tenant
Client Secret	Credential for the application identity
IAM	Controls what the identity is authorized to access
Key Vault	Securely stores secrets and credentials
Secret Scope	Allows Databricks notebooks to retrieve secrets securely
OAuth	Authentication mechanism used by Spark in this setup
ABFSS	Secure filesystem path used to access ADLS Gen2
DataFrame	PySpark representation of the loaded CSV data